

Bachelor Thesis

Reactly: A ReAct inspired Agent from scratch

Author
Donovan Käfer

in partial fulfilment of the requirements for the degree of
Bachelor of Science (BSc)

Vienna, 25.02.2026

Degree programme code:	UA 033 521
Degree programme:	Computer Science - Allgemein
Supervisor:	Dipl.-Ing. Dr.techn Marian Lux

Contents

1 Motivation **4**
- 1.1 Limitations of LLMs . . . 4
  - 1.1.1 Computational demands . . . 4
  - 1.1.2 Context limitations . . . 5
  - 1.1.3 Unreliability and Brittleness . . . 5
- 1.2 Goals of this project . . . 5
  - 1.2.1 Affordable development and deployment . . . 5
  - 1.2.2 Minimal implementation . . . 6
  - 1.2.3 Competitiveness with state-of-the-art alternatives . . . 6

2 Related Work **6**
- 2.1 Chain of Thought (CoT) . . . 6
- 2.2 Reasoning & Acting (ReAct) . . . 7
- 2.3 Chain of Agents (Yusen Zhang et.al. 2024) . . . 8
- 2.4 Chain of Agents (OPPO AI Agent Team) . . . 9
- 2.5 Plan-and-Solve (PS) Prompting . . . 9
  - 2.5.1 Plan-then-Execute . . . 9
- 2.6 Comparison and Discussion of Approaches . . . 10

3 Design **11**
- 3.1 Agent workflow . . . 11
- 3.2 Advantages of task splitting . . . 11
  - 3.2.1 Smaller context sizes . . . 12
  - 3.2.2 Predictability . . . 12

4 Implementation **12**
- 4.1 Code organization . . . 13
  - 4.1.1 agent.py (Agent facade) . . . 13
  - 4.1.2 core.py (Agent workflow) . . . 13
  - 4.1.3 framework.py . . . 14
  - 4.1.4 stages.py . . . 15
  - 4.1.5 types.py . . . 16
- 4.2 LLM interaction . . . 16
- 4.3 Tracing support . . . 17
  - 4.3.1 Tracing API . . . 17
  - 4.3.2 Included Test Tracer . . . 18
- 4.4 Development & source code . . . 18
  - 4.4.1 Git repository . . . 19
  - 4.4.2 PyPi package & Setup . . . 19
  - 4.4.3 Usage of Reactly . . . 20

5 Evaluation **20**
5.1 Evaluation system 21
5.1.1 Argument `-test` (short `-t`) 21
5.1.2 Argument `-agent` (short `-a`) 21
5.1.3 Argument `-llm` (short `-l`) 21
5.2 How to run evaluations 21
5.3 Evaluation results 22
5.3.1 Metrics 22
5.4 HotpotQA results 23
5.5 Custom "fox" results 23
5.6 Summary of results 24

6 Process of Work and Conclusion **24**
6.1 Lessons learned 24

7 Disclosure of AI Usage & Attribution **25**

Abstract

This thesis presents Reactly: A ReAct-inspired agent designed to handle complex tasks by dynamically splitting them into sub-tasks using open-weight LLMs in the 10B parameter range. The task-splitting strategy implemented by Reactly is similar to Plan-then-Execute, enabling separating planning from tool use. Implemented from scratch in 657 lines of code relying on no external dependencies except Ollama, the codebase has been created as a base for customization. The agent is evaluated with both HotpotQA and a custom reasoning test-set. Results show that Reactly can outperform SOTA frameworks such as LlamaIndex's ReActAgent in prediction quality when using significantly more time.

1 Motivation

Large language models (LLMs) have been the flagship application of the recent Artificial Intelligence (AI) boom [16][1]. Being one of the most straightforward applications of the highly successful Transformer [19] architecture, LLMs have not only found their way into our daily lives [2], but one of the most active fields of research has taken up the task of improving, understanding, interpreting, controlling, and making use of them [27].

1.1 Limitations of LLMs

Despite their usefulness, many fundamental limitations of LLMs have become apparent. First, a brief summary of the ones relevant for this work will be given. A summary of the goals of this work intended to tackle them will follow.

1.1.1 Computational demands

Both inference (using a trained model for prediction), but even more so training is typically computationally demanding [16][11][13] and often requires (expensive) dedicated hardware to be run efficiently and fast [11]. Several ways to mitigate these have been found: First of all, ground-up training of new LLMs is done rarely [9], and the same LLMs are used for a variety of different tasks [4][9].

Secondly, there is the move away from LLMs with 100+ billion parameters towards smaller ones with only ∼10 billion parameters or less [18][2]. Usually, these smaller models still need dedicated hardware like modern graphic cards or special AI accelerators as used in smartphones, but make the deployment of LLMs in many more domains (like edge computing [13][24]) possible.

[1] https://www.elon.edu/u/news/2025/03/12/survey-52-of-u-s-adults-now-use-ai-large-language-models-like-chatgpt/, accessed 25.02.2026

[2] https://resourcera.com/data/artificial-intelligence/chatgpt-users, accessed 25.02.2026

1.1.2 Context limitations

One of the most severe limitations of LLMs is the size of the context window [25]. Reasons for this might be found in the superlinear scaling of the attention mechanism in relation to the context window size [19]. Solutions for this problem typically fall into one of two categories: 1. increasing the context window size [5] or 2. reducing the size of the input [25].

Window extension increases the computation demands and might still suffer from an inefficient utilization of the context. This can lead to an issue called "loss-in-the-middle" [14], where the LLM pays less attention to the context further away from beginning and end.

On the other hand, input reduction approaches, such as Retrieval-Augmented Generation (RAG), might fail to include the relevant information in the context. [7][25]

1.1.3 Unreliability and Brittleness

One of the first limitations anyone will experience using LLMs is their tendency to randomly or at least seemingly arbitrary fail in unexpected ways [3][8]. In contrast to traditional non-heuristic software systems that can be designed to behave in a predictable fashion across the range of expected inputs, the LLM's lack of such determinism frequently results in misformatted output and violation of instructions.

Sometimes, just minor or unrelated parts of the prompt can change the LLM's behavior in seemingly arbitrary ways. This phenomenon often referred to as "brittleness" [8] needs to be considered in any agentic architecture. Validation, retry capabilities, and error handling are used to improve agents reliability. [23] [25] [6]

Additionally, the issue of hallucination, where the LLM is making up facts, is one of the most severe problems of LLMs, as it can be more difficult to detect (and correct) than simple formal or syntactical constraints.

1.2 Goals of this project

With the aim to explore techniques to decompose tasks to allow solving them using smaller, less powerful but more widely available models, this objective of this work is to develop and implement a task-splitting agent as both a proof-of-concept, but also a base for further research. In order to meet this objective, a couple of requirements or sub-goals have been defined that are being discussed in the following sections.

1.2.1 Affordable development and deployment

One of the goals of this project is to enable the developed agent to operate on small, locally deployed LLMs. This provides several advantages, including ease of deployment, improved accessibility for research, greater availability, and enhanced data privacy. As about 8 billion parameters (abbreviated as 8B)

is a rough estimate of the performance limit of modern but still affordable hardware [17], this has been defined as the intended model size the agent has been primarily designed for.

Because training LLMs is a performance demanding and effortful process and has been explicitly defined as a non-goal of this project, the agent uses generic and open-weight (sometimes called open-source) models and is therefor "training free". It should be noted that models variants trained to follow instructions should be preferred.

1.2.2 Minimal implementation

The code-base is intended to be kept unusually small to best serve as a base for future development. This improves comprehensibility and maintainability of the code base and allows for more flexible extension and modification.

1.2.3 Competitiveness with state-of-the-art alternatives

While it should be feasible and affordable to develop or deploy the agent and the code base should be comprehensible and flexible, the agent should also be competitive in capability as other common state-of-the-art (SOTA) alternatives such as LlamaIndex[3]. The agent might take more time to execute a query, but deliver higher quality, more correct results provided both use the same LLM on the same hardware and have access to the same pool of tools.

2 Related Work

While these limitations severely restrict plain LLMs in their usefulness and efficiency, many sometimes trivial solutions have been found to avoid these shortcomings.

2.1 Chain of Thought (CoT)

Chain-of-Thought (CoT) [21] is a prompting technique that invokes the language model to write down thoughts within the response before producing a conclusive answer. This technique improves model performance especially on tasks requiring more reasoning where a question cannot be answered trivially just by looking at the question. Domains like mathematics, formal logic, commonsense, and symbolic reasoning are famously hard for plain large language models.

Within CoT, the model is few-shot prompted with a set of exemplary question-answer pairs before the actual question, as made popular by Brown et al. (2020) [4], but with the difference that each exemplary answer includes a chain of spelled-out reasoning steps that lead to the final requested answer. Thus, the

[3]https://www.llamaindex.ai, accessed 25.02.2026

model gets conditioned in an in-context learning fashion to also precede the conclusive answer by a chain of thoughts.

Famously, this technique can improve performance on reasoning-heavy tasks tremendously and is used in many recent architectures. Nevertheless, CoT does not improve on the utilization of the context window and therefore does not help against context degradation or the lost-in-the-middle [14] problem. Other prompting techniques and architectures have built upon this idea, such as tree-of-thoughts [15] (using some form of continuous checking of reasoning steps and backtracking if necessary) and ReAct, which will be discussed below.

2.2 Reasoning & Acting (ReAct)

One of the most powerful techniques for building an agent based on an LLM is called "ReAct", a combination of reasoning and acting [23]. ReAct combines the reasoning capabilities of Chain-of-Thought prompting with the agency coming from the LLM's ability to use provided tools to solve a task or interact with the outside world.

This approach is both very powerful and simple: For each query from the user, the agent loops through a sequence of steps calling the LLM multiple times until the final answer, with which the agent answers the initial query, has been produced. In each loop iteration, the agent performs the steps:

1. Think: Like in Chain-of-Thoughts prompting, the LLM is prompted to output some thoughts into this section, reasoning about the previous context and planning what action to take next. This section will not be processed by the agent code as is only supposed to help the LLM in the next steps.

2. Act: The LLM is prompted again to choose one of the possible actions and generate the corresponding input for the action. Depending on the implementation, the LLM might be called multiple times to chose the action and generate the input separately or it might produce both within one call. The available kinds of actions and the available tools, which (usually) represent a possible action as well, are also implementation dependent. In the typical case where the action will be a chosen tool to be run with some LLM-generated input, the tool will be simply identified by the tool's name and the input will be provided in some easy-to-parse format, like plain text or JSON for example.

3. Perform action: In this step, the agent will call the selected tool and appends the result of the tool call to the context. First, the agent parses the output of the previous Action-section to determine the chosen tool and extract the generated input. Thereafter, it waits for the tool to finish. Lastly, the tool result will be formatted as a so-called "observation" and appended to the context. One special action is Finish (sometimes called "answer"), upon which the provided action input will be returned to the user as answer to the initial question.

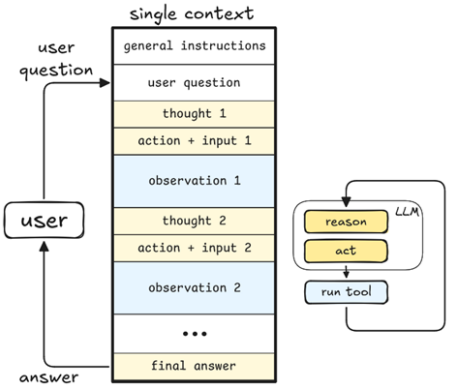


Figure 1: The ReAct loop uses a single context

A single context (or message history) is used as the only working memory of the agent. Each result produced by any step of the sequence, including the initial user question, the responses of the LLM calls (containing the thoughts, chosen actions and action inputs, and answer) and the result of the called tools are being appended to this single context. Each LLM call is prompted with the entire context.

The single-context approach becomes especially problematic when the history includes many tool invocations (as it is often the case for multi-hop reasoning tasks) where the results are huge. This often happens when a retriever-tool is called resulting in large retrieved documents (chunks).

2.3 Chain of Agents (Yusen Zhang et.al. 2024)

This architecture [25] aims to solve the limitation on the context window size by splitting up some long input into chunks and processing each chunk by another worker agent (with its own context). In stage 1, each worker LLM_{W_i} gets prompted with the user query q, it's specific input chunk c_i, some instructions I_W together with a "communication unit" CU from its previous worker agent. The output of one worker is always passed to the next as a CU. In stage 2, a "manager agent" LLM_M uses the CU of the last worker to generate a response to the query, which it is also provided with.

$$CU_i = LLM_{W_i}(I_W, CU_{i-1}, c_i, q)$$

$$Response = LLM_M(I_M, CU_l, q)$$

While this approach addresses the issue with long context coming from a long input (e.g. a long response from a tool or a large file), at least in its original form it lacks the ability to call and use act with tools. It's easy to imagine a modification where each worker agent actually does use some tool, the modified

architecture would still lack the ability to plan the use of the tools across the worker agents.

2.4 Chain of Agents (OPPO AI Agent Team)

Another paper [12] outlines a different approach, which is also called Chain of Agents. Here, an LLM is specifically trained to simulate different sub-agents within its context. Such unified models are referred to as "Agent Foundation Models" (short AFM) by the paper. The AFM might first reason in a `<think>` section of the context, then produce a sequence of steps in a `<plan>` section, then after some thinking calls a tool in a `<tool>` section (which the runtime will append the observation to), and so on, until the model answers the initial question in a final section. The AFM is specifically trained to behave differently while producing different sections of the context, which is called "role playing".

This approach seems very promising and has a number of advantages like the elimination of inter-agent communication overhead. But its requirement for training a model makes it infeasible to do effectively on affordable consumer hardware and is outside the scope of this work. Furthermore, since this approach uses a single context (including the entire results from all previous tool calls), the issue of context window limitation is only made worse.

2.5 Plan-and-Solve (PS) Prompting

The techniques looked at until now think or reason about their following course of action while executing/solving the query. While these single-step reasoning approaches allow for greater flexibility, a paper proposes a technique called "Plan-and-Solve Prompting" (PS) [20], where the LLM is prompted to formulate a plan in advance that is decomposing the entire task into a sequence of subtasks, and then executing the plan to come up with an answer. Zero-shot PS prompting has been shown to outperform zero-shot CoT on arithmetic reasoning tasks.

Although this seems promising for the project goals described earlier, as Plan-and-Solve is still merely a prompting technique, nothing enforces the LLM to strictly follow this pattern and prevents it from deviating in some erroneous, maybe even security relevant ways. The result from a tool (which can easily include malicious content, for example when web-search tools fetch content from external untrusted sites) might manipulate an agent into performing unwanted actions like deleting data or leaking private information through some external tool calls to external parties.

2.5.1 Plan-then-Execute

This lack of predictability and control over the agent lead to a revised architecture that is sometimes called Plan-then-Execute [6] or Plan-and-Execute [4],

[4] LangChain, Plan-and-Execute Agents, `https://blog.langchain.com/planning-agents/` (accessed February 20, 2026).

where the strategic planning and tactical execution are strictly separated.

First the **Planner** module decomposes the user's request into the sequence of sub-tasks. As the user's request can be ambiguous and the quality of the plan is crucial for the success of the agent, very powerful (usually proprietary) LLMs are recommended to use for this stage [6]. The exact form and structure of the produced plan can vary, ranging from simple line-by-line lists in natural language or more strictly formatted JSON lists (as will be used by this project) to more complicated direct acyclic graphs (DAG), that also encode the inter-dependencies between the sub-tasks.

Secondly, the **Executor** carries out the plan, executing the sub-tasks one by one. Crucially, the runtime can decide when, by which model and provided with what data (from what other sub-tasks) a task is executed (or even not executed). This takes back some of the control from the heuristically working model, allowing for greater interpretability, predictability and therefor reliability. For a given task from the plan a specific model best suited for this kind of task might be chosen from a pool of available models.

Finally, the answer to the user query is generated using all, the last or some subset of the intermediate solutions to the tasks. This might be done by another dedicated module or just being done by the last sub-task.

Additionally, according to [6], there might be **Verifier** and **Refiner** modules, both optional. These verify that the generated response answers the user query and possibly refine it.

2.6 Comparison and Discussion of Approaches

Choosing between different approaches, agent architectures and prompting techniques has always to be a question of tradeoffs. Relevant criteria for this decision include predictability [3] and controllability, explainability [26], answer quality and accuracy, performance related criteria such as latency and computational efficiency [11], flexibility and the ability of the agent to recover from errors [6], and lastly, the time, effort and individual experience of implementing these. Some of these, which are especially relevant for the previously mentioned goals, will be discussed below.

While CoT is useful when the task requires complex reasoning, but stays mostly self-contained, ReAct enhances the architecture with the ability to use tools and interact with the environment, making the LLM agentic. Because ReAct iteratively prompts the LLM until the output signals to finish, this approach has a built-in potential to get stuck in loops. While both CoT and ReAct work within a single LLM context, both CoA approaches split the work across different contexts, which is especially helpful for long contexts. Because Plan-then-Execute separates the contexts for planning and work (including tool use), this approach allows for more robust planning less sensitive to tool-outputs.

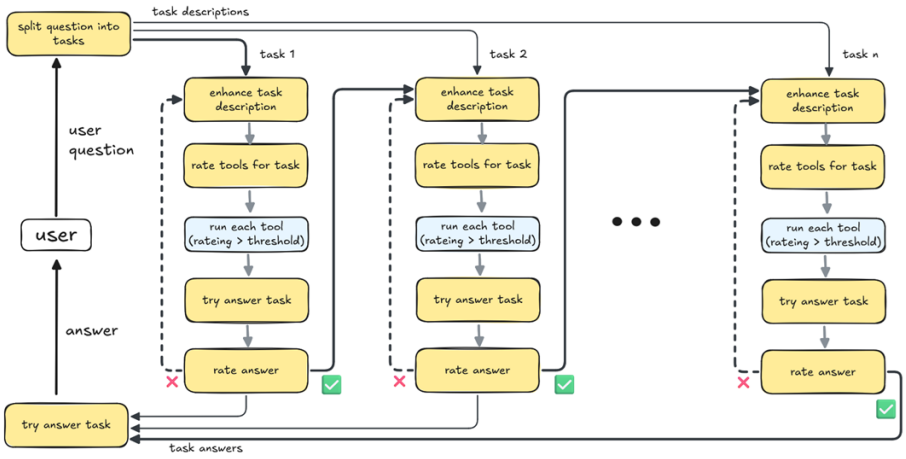


Figure 2: Task splitting in Reactly workflow

3 Design

Now, as multiple different approaches for solving the same key limitations of LLMs have been seen, an introduction to the workflow, that is, when what stage is executed and from where each stage gets its inputs, is given.

3.1 Agent workflow

When a query is made, the agent starts by splitting it into a sequence of sub-tasks using the LLM. Each sub-task is one atomic piece of work needed for answering the query (also called "user question"). Such a task might be determining an atomic fact or calculating some result from already previously information. Solving a single task should not require inter-dependent tool calls, but might take solutions of previous tasks into consideration.

For each task, the agent enhances the task description with relevant information from previous tasks and selects tools from the provided pool suitable for solving this task. Each tool is run with input generated specifically for this tool from the enhanced task description. When all suitable tools are run, the task is answered using the tool results. If the agent is not satisfied with the task answer for the task description, the task is enhanced again using the information contained in the unsatisfactory solution and retried a finite number of times.

Lastly, the agent answers the user question using the solutions to the tasks.

3.2 Advantages of task splitting

Task-splitting is used because of its advantages regarding dealing with large contexts and complicated logic.

3.2.1 Smaller context sizes

Many agent architectures suffer from bloated contexts and its consequential effects. This affects especially agents using some kind of retrieving tool producing verbose output, like web-search or RAG tools. In Reactly, the output of a tool call is only included in the context of a single LLM call in the try-answer-task stage, keeping the agent history compact.

3.2.2 Predictability

One of the core problems of LLMs is their lack of predictability: LLMs are known to make up facts and hallucinate [10], get stuck in (often bad) behavior loops (e.g. redoing the same mistake over and over again) [10], or plainly ignoring given instructions [3]. Such misbehavior seems to be often triggered by a large amount of misleading or erroneous context. For example, a bunch of misformatted JSON in the input might evoke misformatted JSON in the LLM response, even disobeying clear instructions to do otherwise. Although research has shown that it can be challenging to get an LLM to self-correct [10], one common approach to mitigate this issue is to let the LLM do only simple steps with clear instructions. In Reactly, in contrast to many alternative architectures including ReAct, the workflow for answering the query is not a loop iteration until the response of the LLM conforms to some condition (i.e., the "Finish" action in ReAct), but instead determines the set of to be solved tasks at start before handling tool output.

4 Implementation

A prototype agent has been implemented purely in modern Python ($\geq$3.12) with type annotations. It does not use any external dependencies except for `ollama`, which it is based upon. The project source code including the agent was entirely developed from scratch and care was taken to keep the agent small. A mostly procedural coding style making heavy use of data-classes has been followed. Static typing was used throughout the agent for readability and reliability. A modular design was adopted to reduce coupling.

Component	SLOC
`agent (without tracer)`	657
`test tracer`	194
`test tools`	103
`chatbot`	80
`evaluation`	435
`total`	1469

Table 1: Source lines of code per component

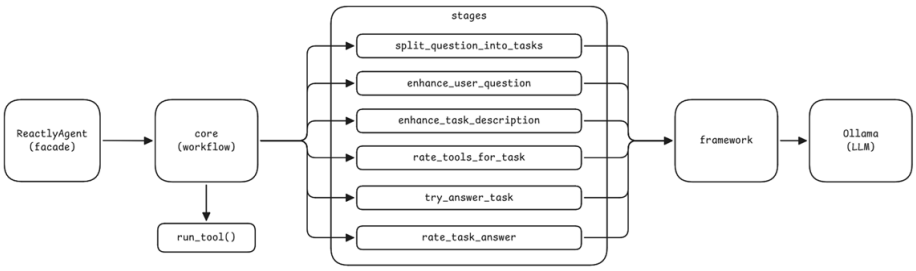


Figure 3: Agent code structure

4.1 Code organization

The entire project has been developed within a single Git repository, as shown in Table 2. The source code implementing the actual agent is provided in the `src/mlux_reactly/` directory, the testing code all resides in `test/`, and `test-files/` contains all test data and files.

4.1.1 agent.py (Agent facade)

`agent.py` implements a small `ReactlyAgent` class that serves as the facade-like interface for the agent. Any code using the agent creates an instance of this class. It includes the agent history, that is, the list of messages (user questions and agent answers) in the order of occurrence. This history is the only state persisting between queries.

Furthermore, the `ReactlyAgent` class also manages the configuration and setup of the agent, including the set of provided tools, the name of the LLM for Ollama, the tracer the agent uses (more to that in a later section) and the `AgentConfig` class specifying various parameters that configures the agent behavior.

Additionally, this file also contains the introspection code wrapping raw Python functions provided as tools into instances of the `tool` class. It uses introspection (via `inspect`) for that.

4.1.2 core.py (Agent workflow)

`core.py` only contains two functions: `run_query()` and `run_tool()`. While `run_tool()` is just a helper dealing with tracing and catching exceptions from the tool, `run_query()` implements the entire workflow of the agent, that is the sequence of LLM-stage and tool calls for answering a user question.

It should be noted that the decision was made to include the entire workflow code within a single function. While splitting up the logic into multiple parts (for example, a "planner" just dealing with task decomposition and a "worker" just solving the sub-tasks) would be usually recommended, because of its small size of only 31 (non-empty) lines of code the benefit of keeping all in one place

`src/mlux_reactly/`	Agent implementation
`tracer/`	Included tracer
`test_tracer.py`	Tracer implementation
`agent.py`	Agent facade
`core.py`	Core workflow of agent
`framework.py`	Abstraction for LLM-call stages
`py.typed`	The project uses typed Python
`stages.py`	LLM-call stages
`types.py`	Types used throughout the agent
`test-files/`	Test files for evaluation
`custom-qa-sets/`	Project specific examples
`hotpot/`	Examples from HotpotQA training set
`rag-docs-default/`	Text-documents for testing
`test/`	Test code
`tools/`	Tools for testing
`calculator.py`	Calculator tool based on `numexpr`
`rag.py`	Retriever tool for local docs
`textcount.py`	Counting characters, words & lines
`websearch.py`	Web-search tool using DuckDuckGo
`wikipedia.py`	Tool for searching the Wikipedia
`chat.py`	Simple chatbot using agent
`eval.py`	Evaluation code
`pyproject.toml`	Project config
`requirements.txt`	Lists external dependencies

Table 2: Project folder structure

outweighs the drawbacks. This not only allows easier adaption of the core workflow to one's needs, but also improves readability.

4.1.3 framework.py

The `framework.py` file provides a layer that abstracts LLM calls. This layer defines the types and provides support for LLM-call stages and the input and output formats used within these stages. Furthermore, the framework generates each prompt in a standardized format and handles the entire interaction between the agent and the LLM using Ollama. This also includes serializing and deserializing of input and output data and dealing with malformatted LLM-responses.

Format descriptions (type `FormatDescr`) define formats of particular data types for use within a prompt. As all data communicated between the agent and LLM is (supposed to be) JSON-encoded, a format is a structure of a JSON value. The `FormatDescr` class includes the template that's used in the "`# Format`" section of a prompt to demonstrate the format to the LLM. The list of `rules` will be added to the prompt of each stage using this `FormatDescr`. The `default` field specifies the value that should be used in the input if the value is not

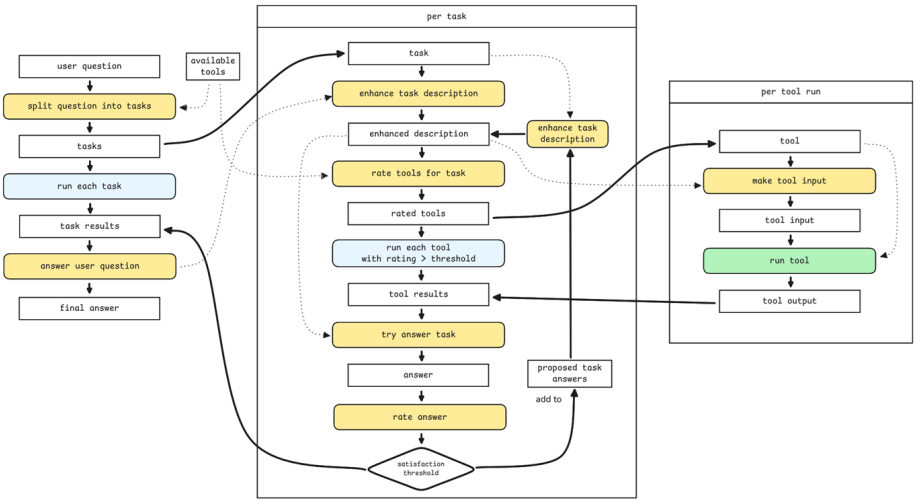


Figure 4: Reactly workflow

provided in the stage call, and the optional preshape function (`preshape_fn`) can be used to convert data into the format before being serialized. Last but not least, each format description also contains a `label` string, that is used both by the agent and the LLM to identify the format and kind of data.

Although format descriptions can be created using the `make_format()` function, the `as_list` method allows for convenient conversion into a list of the same, and the `with_label` method simplifies overwriting the label.

The framework also takes care of tracing, which will be discussed in a later section.

4.1.4 stages.py

All stages and formats are defined within the `stages.py` file. It also contains a few example tools and two helper data-classes: `RatedTool` adding a score to a tool and `ToolRunRecord` for storing the input and output of a tool run.

Using the `make_format` function from the framework, several formats are defined each in a few lines of code. Some formats include a preshape function to shape an input value before being encoded as JSON.

Each stage created using the `make_stage` function defines its name, the properties necessary to generate the prompt, and some additional configuration. The stage name is used mainly for tracing and configuration parameters control the number the LLM call can be retried and specify an alternative result value if the stage failed to produce a valid result every time.

4.1.5 types.py

The main types that are used throughout the agent and partially also exported to the outside are defined in this single place. Types like `Tool`, `LLM`, `Tracer` (including the ineffective `ZeroTracer` implementation) and `AgentConfig` can be used from the outside, `Task`, `TaskResult`, `Answer` and `ChatQA` are only used internally.

4.2 LLM interaction

As the framework-module is the only one interacting with the LLM (through Ollama), it also ensures that both the prompt it calls the LLM with, and the response from the LLM are validly formatted. The format used between the agent and LLM is therefore standardized across all stages of the agent. For the prompt sent to the LLM, a minimal subset of Markdown syntax is used.

The prompt, shown in Listing 1, starts with instructions and rules for the LLM, followed by a Format section demonstrating the used and required formats for the inputs and outputs. The next sections show good (to be followed) and optionally bad (to be avoided) examples. These are formatted the exact same way as real data by the framework. The last section is the `Working Context`, where the inputs are provided. In the very last line, the prompt includes the label for the output, helping the LLM follow to the format and prevent the LLM from responding with anything more than plain JSON.

```
You are a <...designation of the stage...>.
Your job is to <...description what the LLM should do...>

# Rules

- Output valid JSON. No extra text!
- <...additional rules, one per line...>

# Format

The format looks like this:

<Input Label 1>: <JSON-encoded input template line>
<Input Label 2>: <JSON-encoded input template line>
<Output Label>: <JSON-encoded output template line>

# GOOD Examples

<Input Label 1>: <exemplary input data as JSON>
<Input Label 2>: <exemplary input data as JSON>
<Output Label>: <exemplary output data as JSON>

<...there might be multiple examples...>

# BAD Examples
```

```
<Input Label 1>: <exemplary input data as JSON>
<Input Label 2>: <exemplary input data as JSON>
<Output Label>: <exemplary output data as JSON>

# Working Context:

<Input Label 1>: <actual JSON-encoded input data>
<Input Label 2>: <actual JSON-encoded input data>
<Output Label>:
```

Listing 1: Prompt format

The actual runtime data that is inputted to or outputted from the LLM is formatted as a so called "conversation section", where each line starts by the label indicating its type, and is followed by the data encoded as a single line of JSON.

Given the easy with which the framework allows for changing the order of inputs or general layout, multiple things have been tried. Providing the most instructions at the beginning and the shortest input line at the end turned out to be the most successful.

4.3 Tracing support

Often, it can be useful to see what the agent is doing, live while the query is handled or afterward. To support this, the agent has build-in tracing support. On initialization, a tracer is provided to the agent that implements the `Tracer` protocol defined in the agent (in `types.py`). When handling a query, Reactly informs the tracer of possibly relevant events (such as an LLM call) using method calls. It is then the responsibility of the called tracer implementation to deal with the event, for example, by printing a formatted message to `stdout`, logging it to a file or storing it for later. The behavior of the tracer does not affect the agent, except for exceptions raised in the tracer.

4.3.1 Tracing API

The API used between Reactly and the tracer is conceptually based on hierarchical scoping: Every tracer represents a scope of an activity that is performed by the agent. Reactly uses a single tracer instance only for a single activity started by a corresponding event. Every event is of some kind specified by the event `key` and can have any number of attributes, each identified by its name. See Table 3 for a list of event kinds.

Whenever the agent starts with a traceable activity, it calls the `tracer.on()` method of the tracer of the current scope, in which this activity takes place. This method expects a `key` of type `str` and arguments of type `Dict[str, Any]`. After handling the event, `tracer.on()` needs to return a tracer instance for the sub-scope corresponding to this event. Another method called

key	description
`query`	a user query
`task`	subtask the query is split into
`stage`	a stage performing an LLM call
`llmcall`	a single LLM call using Ollama
`toolrun`	invocation of a user-supplied tool
`root`	root node of the tracer
`failed`	a step or operation that failed
`result`	the parent operation finished with some result

Table 3: Event kinds with their keys

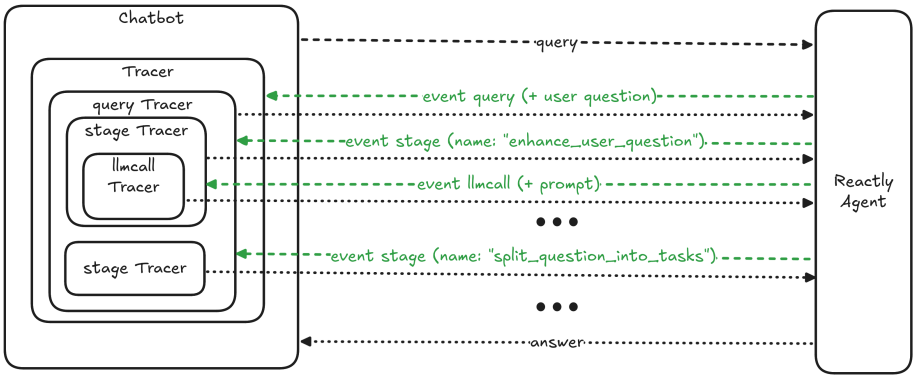


Figure 5: Tracer-Agent interaction

`add_arg(arg_name: str, arg: Any)` can be used by the agent to add arguments to the current scope.

4.3.2 Included Test Tracer

To facilitate easier testing and development, a complete tracer implementation is included within the `mlux-reactly` package in the `tracer` sub-package. Although included, it's not part of the agent, uses the same API and is **not** the default when the `tracer` parameter in not provided in the Reactly constructor call.

4.4 Development & source code

The agent is developed within a single Git repository for version control and provided for use as a Python package using PyPi.

4.4.1 Git repository

The repository is hosted publicly on GitHub under the URL `https://github.com/d-volution/mlux-reactly/`. Readers are encouraged to try it out themselves. Of course, no warranty can be given.

Most commit messages start with a key indicating the kind and scope of the commit (corresponding to the pattern "`key: description of commit`"), such as `feat` for added features or changed behavior, `refactor` for code refactorings, `repo` for repository related changes (e.g. `.gitignore` file), `fix` for bug fixes, `test` for some testing or evaluation code, and names of components of the project such as `textcount tool` or `chat`.

4.4.2 PyPi package & Setup

The agent is also provided as a PyPi[5] package called `mlux-reactly`. Details can be found at `https://pypi.org/project/mlux-reactly/`.

The following describes how to install and use Reactly in an easy to follow way, nevertheless a minimal understanding of Python and the Python ecosystem as well as about Ollama is necessary. It should also be noted that access to hardware with enough performance is needed to run small 8B models.

1. It is strongly recommended to start by setting up and activating a **virtual environment** for your project to isolate its Python packages and configuration from the local setup on the machine. This can be done with the following commands:

   ```
   python3 -m venv .venv
   source .venv/bin/activate
   ```

2. To start, the `mlux-reactly` package containing the agent needs to be installed using the following command:

   ```
   pip install mlux-reactly
   ```

3. **Ollama** has to be installed on the system running the agent. It is recommended to follow the instructions from the README of the Ollama project on GitHub [6].

   The model that should be used needs to be installed as well. As Reactly is designed to work with different LLMs, the decision which one to install can be made by the developer using the agent. The `qwen2.5:7b-instruct-q8_0` model is recommended as the agent was mostly developed and tested with it. The `nomic-embed-text` model is only needed when Reactly is evaluated with the `rag` tool included in the repository (but not included in the agent).

   To install a model, the following command can be used:

[5] https://pypi.org/, accessed 25.02.2026
[6] https://github.com/ollama/ollama/blob/main/README.md, accessed 2026-02-20

```
ollama pull qwen2.5:7b-instruct-q8_0
```

4.4.3 Usage of Reactly

The agent is designed to be used in a very similar way to other SOTA agents, such as from LlamaIndex. When the agent is constructed, a list of tools and the name of the LLM to use can be provided. Additionally, tracing can be set up and a couple of parameters of the agent can be configured.

```
from typing import Annotated
from mlux_reactly import ReactlyAgent, LLM

def count_substr(a: Annotated[str, "Some string"],
    b: Annotated[str, "substring to be counted"]) -> int:
    """How often string b occurs in string a"""
    if not b:
        return 0
    return sum(1 for i in range(len(a) - len(b) + 1)
    if a[i:i+len(b)] == b)

agent = ReactlyAgent(tools=[count_substr],
    llm=LLM("qwen2.5:7b-instruct-q8_0"))

answer = agent.query("How many times does the letter l
    occur in 'artificial general intelligence'?")
print(f"agent answer: {answer}")
```

Listing 2: Usage demonstration of Reactly

As shown in Listing 2, a tool can be created by annotating the function signature using `Annotated` and a doc string. The expected arguments should be of types that can be simply encoded and decoded from JSON, such as `str`, `float`, `int`, `bool` and `lists` or `dicts` of these types. More complicated structures risk confusing small LLMs into generating malformed inputs and therefore the agent successful operation.

The LLM name (which Ollama uses to identify the model) is provided to the agent as a `LLM` class, which is exported from the agent package.

5 Evaluation

To evaluate Reactly in performing as a tool-using agent, an automated evaluation system that can test the agent with a series of different test configurations has been implemented aside the agent. Both Reactly and LlamaIndex's `ReActAgent` were provided the same tools, questions, and hardware for comparability.

5.1 Evaluation system

All files containing test code resides in the `test` directory of the repository, of which `eval.py` is the main evaluation program. The additional files `eval_tracer.py`, `hotpot_based_evaluation.py`, `run_evaluation_qa_file.py`, `test_support.py` and `test_types.py` are supplementary. `llama_index_agent.py` includes adapters posing LlamaIndex agents as Reactly-like agents.

Different tests (including test data), agents, and LLMs (one or multiple for each category) can be defined as an evaluation configuration. The list of `EvalRun`s is the cartesian product of specified tests, agents, and LLMs:

$$[EvalRuns] = [tests] \times [agents] \times [LLMs]$$

The evaluation system uses `argparse` to parse the following arguments:

5.1.1 Argument `-test` (short `-t`)

Currently, the tests `hotpot` (for HotpotQA) and `qa` (QA, for simple Question-Answer sets) are provided. HotpotQA uses for the data-points the same JSON format as the official training set is provided in [7][8].

In addition to the test itself, a "test parameter" can be provided, specifying the test set, and possibly a start and number of examples (data-points) to effectively slice the set. The format of the test argument is `<test-name>/<testset>:<start>:<length>`, for example `hotpot/trainrandom:0:20` for the first 20 examples of a random subset of the training set. Any JSON file in `test-files/hotpot` or `test-files/custom-qa-sets` can be used as a test set in this argument.

5.1.2 Argument `-agent` (short `-a`)

This argument allows the values `reactly` (for the Reactly agent), `llama-react` (for `ReActAgent` of LlamaIndex), and `llama-func` (for the `FunctionAgent` of LlamaIndex). All available agents are listed in `eval.py` in `available_agents`.

5.1.3 Argument `-llm` (short `-l`)

This argument defines with which LLMs the agents should run. For the models, the same names are used as by Ollama.

5.2 How to run evaluations

To run a series of evaluations, the following command can be used:

```
python test/eval.py -test <tests> -agent <agents> -llm <LLMs>
```

[7] `https://github.com/hotpotqa/hotpot`, accessed 20.02.2026

[8] `https://curtis.ml.cmu.edu/datasets/hotpot/hotpot_train_v1.1.json` accessed 20.02.2026

In order to debug the evaluation runs, the evaluation can also be run from within the chatbot `chat.py`:

```
python test/chat.py
>>>> /eval -test <tests> -agent <agents> -llm <LLMs>
```

After the run finishes, the evaluation system stores the result in two files: `evaluation_log_all.jsonl` for the log of all examples including their configuration, question, gold and actual answer, and `evaluation_results.csv` providing statistics about each `EvalRun`, such as average, minimal and maximal query duration, scores taken from HotpotQA (exact match, precision, ...) and more.

5.3 Evaluation results

Although multiple evaluations have been performed throughout the project, two especially expressive ones were chosen to understand the characteristics of the agent.

Note that:

- **qwen2.5:7b** denotes `qwen2.5:7b-instruct-q8_0`, the model Reactly primarily targets.
- **qwen2.5:3b** denotes `qwen2.5:3b-instruct-q4_K_M`, a smaller and differently quantized model.
- **Llama ReAct** denotes LlamaIndex `ReActAgent` [9]

5.3.1 Metrics

The scores EM, F1, recall and precision and the corresponding evaluation scoring code for all evaluations have been adopted from HotpotQA [22].

- **Exact Match (EM)**: how often the answer equals the gold answer (except for punctuation)
- **precision**: how many words of the prediction are correct
- **recall**: how many correct words have been included in prediction
- **F1**: harmonic mean of precision and recall

$$[F1] = 2 \cdot \frac{[precision] \cdot [recall]}{[precision] + [recall]}$$

The column "**time**" denotes the time that the agent took to come up with the response. "**accuracy**" describes the percentage that the answer is correct, evaluated by an LLM-as-a-judge,

[9] `https://developers.llamaindex.ai/python/examples/agent/react_agent/`, accessed 20.02.2026

5.4 HotpotQA results

HotpotQA [22] is a multi-hop question-answer data-set. It has been chosen for this project because it provides a large, diverse dataset but is also used by other previous research, such as ReAct[23].

Agent	LLM	accuracy	EM	F1	prec	recall	time
Reactly	qwen2.5:7b	64%	47%	56%	55%	62%	4.9sec
Llama ReAct	qwen2.5:7b	59%	35%	43%	42%	47%	1.2sec
comparison	qwen2.5:7b	+8%	+34%	+29%	+31%	+31%	4x
Reactly	qwen2.5:3b	10%	8%	10%	10%	9%	1sec
Llama ReAct	qwen2.5:3b	48%	38%	42%	42%	46%	1sec
comparison	qwen2.5:3b	0.21x	-80%	-77%	-76%	-80%	1x

Table 4: Evaluation with comparison of HotpotQA test using trainrandom test-set of size N=100 for different LLMs

As can be seen in Table 4, Reactly clearly outperforms LlamaIndex on `qwen2.5:7b` by about 30%. On the smaller and less capable `qwen2.5:3b` LLM, Reactly actually underperforms in relation to LlamaIndex, which might be caused in the failures of the task-splitting stage to plan ahead well enough, thus breaking the entire workflow.

5.5 Custom "fox" results

For this project, `fox-docs.json`, a small set of 100 fake documents was created by use of ChatGPT[1], densely filled with facts and numbers, that are less likely to appear in any models training data. Then, two sets of question-answer-pairs were created by a combination of AI-suggestion (ChatGPT) and manual adoption: `fox-easy.json` is a dataset of N=50 examples, mostly single-hop requesting a fact from a single document, and `fox-qa.json` (N=20) usually requiring multiple documents to be retrieved. The "fox" evaluation set is made to require a lot of simple calculations with numbers obtainable from the documents, thus showcasing an important use-case of Reactly.

Agent	testset	accuracy	EM	F1	prec	recall	time
Reactly	fox-easy	44%	16%	32%	28%	42%	3.8sec
Llama ReAct	fox-easy	26%	6%	14%	11%	24%	1.2sec
comparison	fox-easy	+69%	2.7x	2.4x	2.5x	+75%	3.2x
Reactly	fox-qa	35%	35%	35%	35%	35%	7.3sec
Llama ReAct	fox-qa	10%	10%	11%	11%	15%	1.3sec
comparison	fox-qa	3.5x	3.5x	3.1x	3.23x	2.3x	6x

Table 5: Evaluation with comparison of QA test-sets using the `qwen2.5:7b` LLM

As shown in Table 5, the Reactly approach can speed up simple logic-heavy tasks (fox-easy) by about 69% in accuracy and about 2.4x in stating correct precise answers (F1). Even more impressive seems the 3.5x improvement for harder multi-hop questions (fox-qa), but it should be noted that the small success rate of LlamaIndex ReActAgent of 10% (for N=20, that means only 2 questions were answered correctly) together with the small number of only 20 examples make this numbers are less reliable.

5.6 Summary of results

There is limited research about the benefits of agents using pre-planned task-splitting in local small-LLM training-free settings. The results show that it is possible for Reactly-like task-splitting to outperform alternative architectures such as ReAct regarding the quality of answers in such settings. As can be seen in the "fox" evaluations, logic-heavy tasks benefit from this approach especially. Although the feasibility of minimalist but competitive designs has been demonstrated, at least for this implementation, it comes with a large cost in runtime.

6 Process of Work and Conclusion

Development for this project started with the implementation of simple chatbots using locally deployed LLMs. These chatbots were soon converted into ReAct architectures, and the simple calculator- and textcount-tool were created for demonstrative use.

Following that, an approach was tried out implementing one form of task-splitting using two instances of the already developed ReAct agent. One instance, the sub-agent, is passed the tools provided to the entire agent, the second, the coordinator (or manager-agent), is only provided a single tool that was created to serve as a sub-agent wrapper: The input provided to this tool is passed on to a sub-agent, it's response is provided as the tool result ("observation").

This approach allows that both agents share the same core-logic code and ReAct loop. Furthermore, each task including their corresponding tool calls would be dealt with in its own context, thus not flooding the main context with irrelevant information.

The reason this approach failed mainly lay in the difficulty of getting the coordinator agent to make sub-tasks and not trying to plainly answer the user question. Following the supervisor's advice, this approach was abandoned in favor of the current architecture using dedicated LLM-call stages.

6.1 Lessons learned

While doing research for this project, a vast and rapidly changing area of AI research has been experienced. As "agentic" AI (a word not even included

in many dictionaries yet) becomes more and more the future of research and development, the prospect and hope confronted with was unique.

Nevertheless, also some challenges were seen: Being a long-time developer trusting software systems in their ability to, if developed and set up correctly, endlessly exactly follow instructions, getting LLMs to do the same felt almost like a Sisyphean task.

However, there were a number of tricks and techniques to make LLMs follow instructions learned. Only using simple Markdown and JSON formatting was equally as much of a help as the supervisor's recommendation to use "instruct" LLMs, that have been trained to do exactly that.

Last but not least, a larger rate of improvement in prediction quality has been expected. Working with small LLMs experiencing their limitations, often failing with simple tasks in unpredicted ways taught both how new this research area is, but also the importance of trial-and-error. It should be obvious that the prediction quality per time even decreased in Reactly compared to LlamaIndex.

Although, at times, the project seemed never-ending, it was an interesting and intense learning experience.

7 Disclosure of AI Usage & Attribution

While this project researches AI, it made heavy use of it as well. ChatGPT [10], Google Gemini[11], Copilot [12] and a few times even Ollama-based LLMs were used to support this work.

- ChatGPT, Gemini were used with hundreds of queries for research. Asked to explain something or find relevant research for some topic, they delivered valuable insights.
- Most of the agent code was written by hand, some exceptions are `describe()` in `agent.py`, and many single- or few-line code snippets in Python. These were written by Gemini, ChatGPT and Copilot.
- Copilot and ChatGPT were often used to debug some issue, both with the LLM behavior, but also regarding the runtime.
- The "fox" evaluation dataset was created by ChatGPT under human supervision.
- ChatGPT and Gemini were used to check and give feedback on LLM prompts and formats. For example, some of the rules were recommended by AI.
- Overleaf (the tool this thesis is written with) has an AI-based bot checking simple wording and grammar.

[10] https://chatgpt.com
[11] https://gemini.google.com/app
[12] https://copilot.microsoft.com/

References

[1] Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Ahmed, I., Islam, S., Datta, P. P., Kabir, I., Chowdhury, N. U. R., and Haque, A. Qwen 2.5: A comprehensive review of the leading resource-efficient llm with potentioal to surpass all competitors. *Authorea Preprints* (2025).

[3] Bowman, S. R. Eight things to know about large language models. *Critical AI 2*, 2 (2024).

[4] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems 33* (2020), 1877–1901.

[5] Chen, S., Wong, S., Chen, L., and Tian, Y. Extending context window of large language models via positional interpolation, 2023. *URL https://arxiv. org/abs/2306.15595* (2023).

[6] Del Rosario, R. F., Krawiecka, K., and de Witt, C. S. Architecting resilient llm agents: A guide to secure plan-then-execute implementations. *arXiv preprint arXiv:2509.08646* (2025).

[7] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., Wang, H., et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997 2*, 1 (2023), 32.

[8] Hergert, L., Berend, G., Szegedy, M., Turan, G., and Jelasity, M. On the brittleness of llms: A journey around set membership. *arXiv preprint arXiv:2511.12728* (2025).

[9] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556 10* (2022).

[10] Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large language models cannot self-correct reasoning yet. *arXiv preprint arXiv:2310.01798* (2023).

[11] Li, R., Fu, D., Shi, C., Huang, Z., and Lu, G. Efficient llms training and inference: An introduction. *IEEE Access 13* (2024), 32944–32970.

[12] Li, W., Lin, J., Jiang, Z., Cao, J., Liu, X., Zhang, J., Huang, Z., Chen, Q., Sun, W., Wang, Q., et al. Chain-of-agents: End-to-end agent foundation models via multi-agent distillation and agentic rl, 2025. *URL https://arxiv.org/abs/2508.13167 2* (2025).

[13] Liu, C., and Zhao, J. Resource allocation for stable llm training in mobile edge computing. In *Proceedings of the Twenty-fifth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing* (2024), pp. 81–90.

[14] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts, 2023. *URL https://arxiv.org/abs/2307.03172 2* (2023).

[15] Long, J. Large language model guided tree-of-thought. *arXiv preprint arXiv:2305.08291* (2023).

[16] Samsi, S., Zhao, D., McDonald, J., Li, B., Michaleas, A., Jones, M., Bergeron, W., Kepner, J., Tiwari, D., and Gadepally, V. From words to watts: Benchmarking the energy costs of large language model inference. In *2023 IEEE high performance extreme computing conference (HPEC)* (2023), IEEE, pp. 1–9.

[17] Song, Y., Mi, Z., Xie, H., and Chen, H. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (2024), pp. 590–606.

[18] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[19] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems 30* (2017).

[20] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (2023), pp. 2609–2634.

[21] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems 35* (2022), 24824–24837.

[22] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing* (2018), pp. 2369–2380.

[23] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations* (2022).

[24] Zhang, M., Shen, X., Cao, J., Cui, Z., and Jiang, S. Edgeshard: Efficient llm inference via collaborative edge computing. *IEEE Internet of Things Journal 12*, 10 (2024), 13119–13131.

[25] Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., and Arik, S. O. Chain of agents: Large language models collaborating on long-context tasks, 2024. *URL https://arxiv.org/abs/2406.02818 3* (2024).

[26] Zhao, H., Chen, H., Yang, F., Liu, N., Deng, H., Cai, H., Wang, S., Yin, D., and Du, M. Explainability for large language models: A survey. *ACM Transactions on Intelligent Systems and Technology 15*, 2 (2024), 1–38.

[27] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. A survey of large language models. *arXiv preprint arXiv:2303.18223 1*, 2 (2023), 1–124.